# 5 Fundamentação Teórica

Este capítulo apresenta a base conceitual necessária para o desenvolvimento da arquitetura metodológica do projeto. O escopo abrange desde os fundamentos da Geração Procedural de Conteúdo (PCG), passando pelas capacidades e limitações espaciais dos Modelos de Linguagem de Grande Porte (LLMs), até culminar no estado da arte das arquiteturas híbridas neuro-simbólicas. Tais conceitos são indispensáveis para justificar a delegação de responsabilidades entre o modelo generativo – que atua como arquiteto – e o motor de jogo determinístico, que atua como engenheiro validador das restrições espaciais tridimensionais.

## *5.1 Geração Procedural de Conteúdo (PCG) e Machine Learning (PCGML)*

A Geração Procedural de Conteúdo (PCG - *Procedural Content Generation*) é uma subárea fundamental na pesquisa de Inteligência Artificial aplicada a jogos. Historicamente, PCG refere-se à criação algorítmica de dados do jogo (como níveis, narrativas, texturas e regras) de maneira autônoma ou semi-autônoma, muitas vezes combinada à aleatoriedade para garantir variabilidade e rejogabilidade. Tradicionalmente, as abordagens de PCG podiam ser categorizadas como métodos construtivos (como autômatos celulares e gramáticas de forma) ou métodos baseados em busca (algoritmos genéticos e otimização evolucionária) (Summerville *et al.*, 2018). No que tange aos métodos clássicos, como destacado na literatura de geração procedural (Vieira, 2025), a principal vantagem reside na garantia de resolução: uma vez que as regras determinísticas de particionamento de espaço (como as encontradas na estruturação modular baseada em *tiles*) são definidas, a saída estrutural é previsível. Contudo, essa abordagem de PCG pura esbarra na falta de semântica criativa; os geradores construtivos produzem artefatos funcionalmente corretos, mas frequentemente carecem de uma narrativa ou intenção arquitetônica clara. Para contornar a rigidez dos métodos construtivos tradicionais, emergiu a vertente *Procedural Content Generation via Machine Learning* (PCGML). Segundo o trabalho fundamental de (Summerville *et al.*, 2018), o PCGML propõe treinar modelos a partir de uma base de dados estruturada de conteúdo de jogos existentes. Ao contrário dos algoritmos baseados em busca, os quais geram níveis testando-os através de uma função de aptidão (*fitness function*), o modelo treinado gera o conteúdo diretamente emulando os padrões latentes no corpus de dados originais. Inicialmente limitados a modelos estatísticos mais modestos (como cadeias de Markov bidimensionais ou LSTMs), a ascensão dos grandes modelos fundacionais abriu portas para uma formulação textual mais declarativa para a criação dos mundos. É sob essa ótica evolutiva do PCGML que o uso atual de Modelos de Linguagem de Grande Porte (LLMs) se apoia (Gallotta *et al.*, 2024).

### *5.2 O Problema da Ancoragem Espacial (Spatial Grounding) em LLMs*

Os Modelos de Linguagem de Grande Porte (LLMs) revolucionaram diversas esferas da automação e da IA generativa por meio de redes neurais baseadas na arquitetura *Transformer*. Eles demonstram proficiência notável no raciocínio estruturado, planejamento narrativo e na manutenção de coerência temática. Sua capacidade inata de entender dependências lógicas os torna ideais para desenhar fluxos de progressão, gerando rapidamente esquemas semânticos formatados, por exemplo, como estruturas JSON. No entanto, o paradigma do treinamento em PLN (Processamento de Linguagem Natural), que é estritamente baseado no sequenciamento unidimensional (1D) de *tokens*, introduz sérias desvantagens quando aplicado em contextos espaciais tridimensionais (3D). Pesquisas recentes de metodologias aplicadas apontam a carência de *spatial grounding* (ancoragem física e espacial) como a principal barreira técnica enfrentada pelos modelos generativos atuais em domínios arquitetônicos de jogos. Estudo exploratório sobre o *workflow* do level designer perante a IA indica que LLMs são excepcionalmente propensos a cometer "alucinações geométricas"em arranjos 3D (Re, 2024). Sem estar amparado a uma verificação geométrica contínua, o LLM falha ao dimensionar distâncias, prever áreas de secção transacional e posicionar elementos num *grid* tridimensional complexo. Tais achados são sustentados de maneira recorrente pela literatura do estado da arte. Por exemplo, trabalhos como o *Code-as-Room* (Yang *et al.*, 2026) e o estudo de (Feng *et al.*, 2026) acerca da geração espacial baseada em layout autorregressivo demonstram evidências empíricas de que a geração puramente pautada na inferência probabilística leva inevitavelmente ao *overlap* de *bounding boxes* (intersecção e sobreposição de malhas) e ao deslocamento de portas ou vias cruciais para áreas inacessíveis. (Sun *et al.*, 2025) chegam a conclusões semelhantes em seu estudo sobre planejadores explícitos: modelos de texto puro não compreendem instintivamente as propriedades da geometria euclidiana, frequentemente posicionando mobiliário flutuando e gerando cômodos cujas portas desembocam no vácuo ou se conflitam. Portanto, essas limitações inviabilizam o uso "direto"de um LLM como motor que edita livremente um espaço virtual.

### *5.3 Modelagem de Fases via Grafos de Progressão*

Cientes da deficiência dos LLMs no espaço e sua eficácia na lógica semântica pura, autores vêm recorrendo à teoria dos grafos. A estruturação de fases em *Level Design* pode ser separada em dois paradigmas: o da "topologia"e o da "topografia"(espaço contínuo e dimensão física). Topologicamente, uma fase inteira de um jogo digital (como uma masmorra ou um complexo labiríntico) pode ser abstratamente resumida a um grafo relacional ou grafo de progressão lógico (Chen; Jhala, 2025). Nesse grafo, os vértices (nós) representam as salas, pátios ou instâncias temáticas e os nós direcionais ou arestas delineiam a conectividade (portas, bloqueios lógicos ou teletransportes). Esta representação abstrai completamente as coordenadas flutuantes ou as restrições tridimensionais, aproximando-se perfeitamente da expertise linguística de um LLM (Gallotta *et al.*, 2024). Delegar a criação

de grafos relacionais (como uma cadeia estruturada de chaves JSON determinando o nó A conecta-se ao nó B condicionalmente) ao invés do mapeamento métrico do espaço resolve a incapacidade da Inteligência Artificial em operar sobre a geometria. Este modelo de divisão em grafos viabiliza que a ferramenta generativa se limite à elaboração do ritmo do percurso e ao arranjo sistêmico das recompensas e ameaças, blindando a arquitetura do projeto de alucinações matemáticas espaciais.

## *5.4 Arquiteturas Híbridas e Lógica Determinística*

A síntese entre o poder generativo de um LLM com a confiabilidade rígida do processamento clássico resultou na concepção de sistemas neuro-simbólicos ou *pipelines* de arquiteturas híbridas. Na literatura atual envolvendo PCG avançado, esta dicotomia costuma ser referida pelo paradigma Arquiteto-Engenheiro ou "Dual-Agent", sendo amplamente considerada o estado da arte para sínteses em 3D. Frameworks baseados neste paradigma delegam ao LLM estritamente a tarefa de ideação ou macro-planejamento semântico (*Arquiteto*), enquanto injetam essas descrições simbólicas para um validador algorítmico tradicional (*Engenheiro*). A execução e instanciamento destas diretrizes se valem de programação estrita baseada em restrições lógicas (*Constraints*) (Xu; Verbrugge, 2025b). Trabalhos como *SceneCraft* (Hu *et al.*, 2024), voltados à síntese programática no Blender, bem como implementações direcionadas à interpretação de cenas em jogos virtuais (Xu; Verbrugge, 2025a), utilizam explicitamente LLMs apenas para a estrutura narrativa base, deixando a manipulação do mundo e a aplicação de coordenadas matemáticas sob estrito controle algorítmico. Em contextos equivalentes, (Her *et al.*, 2025) exploram a eficácia da abordagem de agência dupla (*Dual-Agent Architecture*) para mapas 3D *Zero-shot*, onde um agente projeta os requisitos criativos da cena e um outro atua como validador implacável de regras operacionais em uma *game engine*. Da mesma forma, propostas de metodologias narrativas para cenários procedurais (como o modelo *Word2World* (Nasir; James; Togelius, 2024)) confiam em *strings* de texto abstratas processadas e transcodificadas deterministicamente por uma *engine* em tempo de execução para gerar instâncias 2D ou 3D robustas e livres de erro de colisão. A premissa central e o embasamento dessa arquitetura são de que, por trás do ruído natural do aprendizado não determinístico de máquina, haja sempre um filtro algorítmico rigoroso em código tradicional para sustentar a interface com o jogador.

## *5.5 Testes de Restrições geométricas e integridade de navegação*

Na camada inferior do paradigma híbrido – que no projeto é encapsulado pelas lógicas em C# ou GDScript rodando nativamente sobre as ferramentas do Godot Engine – a abstração oriunda do JSON transforma-se em vértices instanciáveis. Motores de jogo modernos detêm subsistemas potentes, particularmente motores de física dotados da habilidade de computar *Axis-Aligned Bounding Boxes* (AABBs), hierarquias em volumes limitantes e lançamento de raios espaciais (*Raycasting*). Tais operações compõem a base

algorítmica capaz de varrer o plano matemático tridimensional, impedindo sobreposições visuais indesejáveis de malhas (*Meshes*). Em abordagens mistas aplicadas no mercado, a participação humana na interface PCG necessita atuar numa camada de testes estritos como forma de garantir uma disposição geométrica plausível (Walton; Rahat; Stovold, 2022). O principal subproduto gerado pelas restrições geométricas, além de assegurar que módulos modulares ou corredores pré-fabricados não intercedam no mesmo polígono de forma impenetrável, é o cálculo da jogabilidade efetiva, muitas vezes verificada na computação da Malha de Navegação (*Navigation Mesh* ou *NavMesh*). Uma NavMesh atua mapeando de forma conexa todos os espaços fisicamente percorríveis de um ambiente virtual para atores virtuais ou controladores de jogador. O projeto de um jogo não detém o aval do sucesso construtivo se instâncias procedurais falharem no isolamento topológico; caminhos traçados que induzem *dead-ends* imprevisíveis ou rotas não conectadas condenam toda a geração. É exigido que o algoritmo do motor atue aplicando verificações iterativas na fase gerada: se uma sobreposição violar o percurso até o nó de saída estipulado no JSON inicial, algoritmos complementares de resolução de caminhos, como o famoso $A*$, atuarão identificando um bloqueio insolúvel. Tais abordagens teóricas de integridade de malha referendam e justificam a adoção do motor de jogo não apenas como visualizador, mas como a última barreira formal da validação da coerência entre o plano abstrato (IA) e o plano funcional (Jogo) que a pesquisa objetiva integrar.

## Bibliografia

CHEN, Y.-C.; JHALA, A. **Narrative-to-Scene Generation: An LLM-Driven Pipeline for 2D Game Environments**. [*S. l.: s. n.*], 2025. arXiv: 2509.04481 [cs.GR]. Disponível em: https://arxiv.org/abs/2509.04481.

FENG, H. *et al.* **Repurposing 3D Generative Model for Autoregressive Layout Generation**. [*S. l.: s. n.*], 2026. arXiv: 2604.16299 [cs.CV]. Disponível em: https://arxiv.org/abs/2604.16299.

GALLOTTA, R. *et al.* Large Language Models and Games: A Survey and Roadmap. **IEEE Transactions on Games**, Institute of Electrical e Electronics Engineers (IEEE), p. 1–18, 2024. ISSN 2475-1510. DOI: 10.1109/tg.2024.3461510. Disponível em: http://dx.doi.org/10.1109/TG.2024.3461510.

HER, L. C. *et al.* **Zero-shot 3D Map Generation with LLM Agents: A Dual-Agent Architecture for Procedural Content Generation**. [*S. l.: s. n.*], 2025. arXiv: 2512.10501 [cs.AI]. Disponível em: https://arxiv.org/abs/2512.10501.

HU, Z. *et al.* **SceneCraft: An LLM Agent for Synthesizing 3D Scene as Blender Code**. [*S. l.: s. n.*], 2024. arXiv: 2403.01248 [cs.CV]. Disponível em: https://arxiv.org/abs/2403.01248.

NASIR, M. U.; JAMES, S.; TOGELIUS, J. **Word2World: Generating Stories and Worlds through Large Language Models**. [*S. l.: s. n.*], 2024. arXiv: 2405.06686 [cs.CL]. Disponível em: https://arxiv.org/abs/2405.06686.

RE, F. Integration of Intelligent agents in Game Development: Prompt Engineering Methodologies for Automatic code generation, 2024.

SUMMERVILLE, A. *et al.* **Procedural Content Generation via Machine Learning (PCGML)**. [*S. l.: s. n.*], 2018. arXiv: 1702.00539 [cs.AI]. Disponível em: https://arxiv.org/abs/1702.00539.

SUN, W. *et al.* **RoomPlanner: Explicit Layout Planner for Easier LLM-Driven 3D Room Generation**. [*S. l.: s. n.*], 2025. arXiv: 2511.17048 [cs.CV]. Disponível em: https://arxiv.org/abs/2511.17048.

TANG, S. *et al.* UnrealLLM: Towards Highly Controllable and Interactable 3D Scene Generation by LLM-powered Procedural Content Generation. *In:* CHE, W. *et al.* (ed.). **Findings of the Association for Computational Linguistics: ACL 2025**. [*S. l.*]: Association for Computational Linguistics, jul. 2025. p. 19417–19435. DOI: 10.18653/v1/2025.findings-acl.994. Disponível em: https://aclanthology.org/2025.findings-acl.994/.

TODD, G. *et al.* Level Generation Through Large Language Models. *In:* PROCEEDINGS of the 18th International Conference on the Foundations of Digital Games. [*S. l.*]: ACM,

abr. 2023. (FDG 2023), p. 1–8. DOI: 10.1145/3582437.3587211. Disponível em: http://dx.doi.org/10.1145/3582437.3587211.

VIEIRA, A. H. G. **Metodologia de geração procedural de conteúdo para estruturas narrativas em jogos eletrônicos**. [*S. l.: s. n.*], fev. 2025. Disponível em: https://hdl.handle.net/11449/295661.

WALTON, S. P.; RAHAT, A. A. M.; STOVOLD, J. Evaluating Mixed-Initiative Procedural Level Design Tools Using a Triple-Blind Mixed-Method User Study. **IEEE Transactions on Games**, Institute of Electrical e Electronics Engineers (IEEE), v. 14, n. 3, p. 413–422, set. 2022. ISSN 2475-1510. DOI: 10.1109/tg.2021.3086215. Disponível em: http://dx.doi.org/10.1109/TG.2021.3086215.

XU, K.; VERBRUGGE, C. **A Database-Driven Framework for 3D Level Generation with LLMs**. [*S. l.: s. n.*], 2025. arXiv: 2508.18533 [cs.AI]. Disponível em: https://arxiv.org/abs/2508.18533.

XU, K.; VERBRUGGE, C. Constraint Is All You Need: Optimization-Based 3D Level Generation with LLMs. *In:* PROCEEDINGS of the 20th International Conference on the Foundations of Digital Games. [*S. l.*]: Association for Computing Machinery, 2025. (FDG '25). DOI: 10.1145/3723498.3723840. Disponível em: https://doi.org/10.1145/3723498.3723840.

YANG, Y. *et al.* **Code-as-Room: Generating 3D Rooms from Top-Down View Images via Agentic Code Synthesis**. [*S. l.: s. n.*], 2026. arXiv: 2605.18451 [cs.CV]. Disponível em: https://arxiv.org/abs/2605.18451.